Activity 5.1: Teaching Suggestion Logic

1. Activity Overview & Objective

In this activity, instructor-facing teaching suggestions are generated using engagement trends derived from the multimodal agent. Temporal engagement profiles are analyzed to detect classroom patterns such as sustained disengagement, attention drops, high-interaction periods, and noisy classroom conditions.

> **Decision-Support Philosophy:** Rule-based logic is applied to convert detected temporal patterns into **context-aware recommendations**. The teaching suggestion mechanism is designed to support instructional decision-making rather than replace it. All suggestions are grounded in observable data trends and presented in a non-prescriptive manner, ensuring transparency and trust.

2. Pattern-to-Recommendation Mapping Matrix

By continuously evaluating the temporal engagement profile and multimodal indicators, the engine categorizes classroom dynamics into structured notification tiers (`alert`, `warning`, and `info`) paired with actionable, non-prescriptive cues:

Detected Temporal Pattern	Observable Data Trend	Signal Type	Context-Aware Recommendation
Individual Attention Drop	Student attention metric drops below 40% threshold (`p['attention'] < 40`).	`alert`	Flags specific student distraction level so the instructor can subtly re-engage them.
Sustained Disengagement	Global class average attention falls below 60% across active participants (`avg_att < 60`).	`warning`	Suggests introducing interactive activities, polls, or a brief Q&A segment to restore focus.
High-Interaction / Peak Period	Sustained high engagement (≥ 85%) and balanced emotional diversity across sessions.	`info`	Highlights effective teaching segments and strengths to reinforce successful pacing.
Noisy Classroom Conditions	Elevated ambient acoustic energy without structured turn-taking or visual focus.	`warning`	Flags classroom management cues to help transition the room back to structured tasks.

3. Backend Implementation

The teaching suggestion architecture is implemented across two core backend services: `TeacherProfileService` (`backend/core/teacher_profiles.py`) for aggregating longitudinal effectiveness, session history, and student trends, and `RecommendationsEngine` (`backend/core/recommendations_engine.py`) for immediate rule-based classroom feedback.

3.1 Longitudinal Instructor & Trend Aggregation (`backend/core/teacher_profiles.py`)

```
# File: backend/core/teacher_profiles.py
from sqlalchemy.orm import Session
from .database import SessionLocal, Session as SessionModel
import random

class TeacherProfileService:
    def get_profile(self, teacher_id: str):
        # Mock calculation - in production, aggregate real session data
        # Real logic: fetch average attention from all sessions by this teacher
        return {
            "teacher_id": teacher_id,
            "effectiveness_score": 87.5,
            "trend": 12.0, # +12%
            "class_history": [
                {"name": "CS101 (Today)", "engagement": 85},
                {"name": "CS101 (Yesterday)", "engagement": 78},
                {"name": "Math Workshop", "engagement": 92}
            ],
            "student_trends": [
                {"name": "Priya", "change": 15, "status": "improved"},
                {"name": "Kumar", "change": 8, "status": "improved"},
                {"name": "Asha", "change": -5, "status": "declining"}
            ],
            "strengths": [
                "High engagement sessions (85%+)",
                "Emotion diversity management"
            ]
        }
```

3.2 Real-Time Rule-Based Recommendation Engine (`backend/core/recommendations_engine.py`)

```
# File: backend/core/recommendations_engine.py
class RecommendationsEngine:
    def generate_realtime_recommendations(self, session_status: dict):
        """
        Rule-based recommendations for immediate feedback.
        Input: Session status dict (people, emotions, attention)
        Output: List of { type: 'alert'|'warning'|'info', message: str }
        """
        recs = []

        # 1. Low Attention Alert
        avg_att = 0
        people = session_status.get('people', [])
        if people:
            avg_att = sum(p['attention'] for p in people) / len(people)

            # Individual check
            for p in people:
                if p['attention'] < 40:
                    recs.append({
                        'type': 'alert',
                        'message': f"Student {p['id']} is distracted
({p['attention']}%)."
                    })

        # 2. Global Attention Warning
        if len(people) > 2 and avg_att < 60:
            recs.append({
                'type': 'warning',
                'message': "Class attention is dropping (<60%). Consider an
interactive activity."
            })

        return recs
```

3.3 Key Engineering Highlights

- **Observable Data Grounding:** Every generated alert or warning traces directly to measurable session metrics (individual attention percentages, global class averages, and longitudinal trend deltas).
- **Non-Prescriptive Instructor Support:** By separating `alert`, `warning`, and `info` cues and pairing real-time telemetry with historical strengths in `TeacherProfileService`, instructors retain full pedagogical autonomy.